

LAPORAN PRATIKUM PEMOGRAMAN BERORIENTASI OBJEK

“CLASS, OBJECT, CONSTRUCTOR dan METHOD”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH PEMOGRAMAN
BERORIENTASI OBJEK

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM.

ASISTEN PRAKTIKUM ; EZZA ADDINI

FAKULTAS TEKNOLOGI
INFORMASI DEPARTEMEN
INFORMATIKA UNIVERSITAS
ANDALAS

2026

A. Langkah Kerja Praktikum

a. Membuat Class Rekening

1. Buat class Rekening

```
public class Rekening {
```

Langkah ini membuat class Rekening sebagai model dasar. Class berfungsi sebagai cetakan yang nantinya digunakan untuk membuat banyak objek rekening dengan data yang berbeda-beda.

2. Tambahkan State/Atribut

```
String nomorRekening;  
String namaPemilik;  
double saldo;
```

Bagian ini mendefinisikan atribut atau variabel instance yang wajib dimiliki setiap objek rekening, yaitu nomor rekening, nama pemilik, dan saldo. Setiap objek yang dibuat dari class ini akan memiliki nilai atributnya masing-masing, terpisah satu sama lain.

3. Buat Constructor

```
public Rekening(String nomor, String nama, double saldoAwal) {  
    nomorRekening = nomor;  
    namaPemilik = nama;  
    saldo = saldoAwal;  
    System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);  
}
```

Constructor berfungsi untuk menginisialisasi nilai atribut pada saat objek baru dibuat menggunakan `new Rekening(...)`. Dengan adanya constructor, data rekening langsung terisi begitu objek dibuat, tanpa perlu diatur satu per satu secara manual. Baris `println` di akhir digunakan sebagai konfirmasi bahwa rekening berhasil dibuat.

4. Buat Method/Behavior

```

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("-----");
}

```

Jika atribut merepresentasikan data yang dimiliki objek, method merepresentasikan perilaku atau aksi yang dapat dilakukan objek tersebut. Method `setorTunai` melakukan validasi nominal (harus lebih dari 0) sebelum menambahkannya ke saldo, untuk mencegah input yang tidak valid. Method `cekInformasi` digunakan untuk menampilkan seluruh data rekening kepada pengguna.

b. Membuat Engine CLI pada Main.java

1. Buat class Main dan method main

```

public class Main {

    public static void main(String[] args) {

```

Class Main berfungsi sebagai tempat program dijalankan. Method main merupakan titik masuk (entry point) wajib pada program Java, sehingga eksekusi program akan dimulai dari method ini.

2. Buat Scanner untuk membaca input

```

Scanner input = new Scanner(System.in);

```

Objek Scanner digunakan agar program dapat menerima input dari pengguna, seperti pilihan menu dan nominal transaksi.

3. Buat variabel penyimpanan objek Rekening

```

Rekening akunAktif = null; // Objek belum diinisialisasi (null)
boolean isRunning = true;

```

Variabel `akunAktif` digunakan untuk menyimpan referensi ke objek Rekening yang sedang digunakan, dan diinisialisasi null karena pada

awal program belum ada rekening yang dibuat. Variabel `isRunning` digunakan sebagai kondisi kontrol perulangan menu.

4. Buat perulangan menu

```
while (isRunning) {  
    System.out.println("\nMenu Utama:");  
    System.out.println("1. Buka Rekening Baru");  
    System.out.println("2. Setor Tunai");  
    System.out.println("3. Tarik Tunai");  
    System.out.println("4. Cek Informasi Rekening");  
    System.out.println("5. Ganti Akun");  
    System.out.println("0. Keluar");  
    System.out.println("Pilih menu: ");  
  
    int pilihan = input.nextInt();  
    input.nextLine(); // Membersihkan buffer enter  
}
```

Perulangan `while` digunakan agar menu terus ditampilkan berulang kali sampai pengguna memilih untuk keluar. Baris `input.nextLine()` setelah `input.nextInt()` diperlukan untuk membersihkan karakter newline sisa input, agar input string pada baris berikutnya tidak terlewat.

5. Implementasikan setiap pilihan menggunakan switch

```

switch(pilihan) {
    case 1:
        System.out.print("Masukkan No Rekening: ");
        String no = input.nextLine();
        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();
        System.out.print("Masukkan Saldo Awal: ");
        double saldo = input.nextDouble();
        input.nextLine();

        // Instansiasi object / Menjalankan constructor
        Rekening rekeningBaru = new Rekening(no, nama, saldo);
        daftarRekening.add(rekeningBaru); // Tambahkan ke ArrayList, tidak menimpa yang lama
        akunAktif = rekeningBaru; // Otomatis jadi akun aktif setelah dibuat
        break;
    case 2:
        if (akunAktif == null) {
            System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor); // Memanggil Behavior / method
        }
        break;
    case 3:
        if (akunAktif == null) {
            System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening");
        } else {
            System.out.print("Masukkan nominal tarik: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        }
        break;
    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;
    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
}

```

Struktur switch mengarahkan program ke blok kode sesuai pilihan menu yang diinput pengguna. Pada menu 2, 3, dan 4 (setor, tarik, cek info), dilakukan pengecekan `akunAktif == null` terlebih dahulu untuk memastikan rekening sudah tersedia sebelum transaksi dilakukan, guna mencegah error.

6. Tutup scanner di akhir program

```

input.close();

```

Setelah perulangan selesai (pengguna memilih keluar), objek Scanner ditutup untuk melepaskan resource yang digunakan.

B. Latihan / Tugas Praktikum

a. Implementasi Fitur Tarik Tunai

1. Tambahkan method tarikTunai pada class Rekening

```
public void tarikTunai(double nominal) {  
    if (nominal <= 10000) {  
        System.out.println("Transaksi Gagal: Minimal nominal penarikan 10.000");  
    } else if (nominal > saldo) {  
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp." + saldo );  
    } else {  
        saldo = saldo - nominal;  
        System.out.println("Tarik tunai Rp" + nominal + " berhasil");  
        System.out.println("Saldo anda sekarang: Rp." + saldo);  
    }  
}
```

Method ini ditambahkan pada class Rekening. Method ini menggunakan struktur if-else if-else untuk menjalankan dua tahap validasi sebelum saldo benar-benar dikurangi:

- Kondisi pertama ($\text{nominal} \leq 10000$) memeriksa apakah nominal penarikan memenuhi batas minimum. Jika tidak, transaksi ditolak dan saldo tidak berubah.
- Kondisi kedua ($\text{nominal} > \text{saldo}$) memeriksa apakah saldo yang dimiliki mencukupi untuk penarikan tersebut. Jika nominal melebihi saldo, transaksi juga ditolak.
- Jika kedua kondisi tersebut tidak terpenuhi (nominal valid dan saldo mencukupi), maka blok else dijalankan: saldo dikurangi sejumlah nominal, kemudian ditampilkan pesan konfirmasi beserta saldo terbaru.

Urutan pengecekan ini penting, karena validasi nominal minimum dan validasi kecukupan saldo harus dilakukan sebelum operasi pengurangan saldo dieksekusi, agar tidak terjadi transaksi dengan nilai yang tidak valid.

2. Lengkapi case 3 pada switch menu di class Main

```

case 3:
    if (akunAktif == null) {
        System.out.println("Error : Mohon maaf, Anda belum memiliki nomor rekening");
    } else {
        System.out.print("Masukkan nominal tarik: ");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;

```

Blok ini ditambahkan pada struktur switch di method main, sebagai penghubung antara menu yang dipilih pengguna dengan method tarikTunai yang telah dibuat pada class Rekening. Sebelum meminta input nominal, dilakukan pengecekan `akunAktif == null` terlebih dahulu untuk memastikan pengguna sudah memiliki rekening, guna mencegah error apabila menu ini dipilih sebelum ada rekening yang dibuat. Jika `akunAktif` tersedia, program meminta input nominal penarikan, kemudian memanggil `akunAktif.tarikTunai(tarik)` untuk menjalankan proses penarikan pada objek rekening.

b. Multi-Akun

1. Import ArrayList

```
import java.util.ArrayList;
```

Ditambahkan di baris paling atas file Main.java. Import ini diperlukan agar class ArrayList dari package java.util dapat digunakan sebagai struktur data penyimpanan.

2. Tambahkan variabel ArrayList untuk menyimpan seluruh rekening

```
ArrayList<Rekening> daftarRekening = new ArrayList<>(); // Menyimpan semua rekening
```

Ditambahkan di dalam method main, sejajar dengan deklarasi `akunAktif`. Variabel ini berfungsi sebagai wadah untuk menampung seluruh objek Rekening yang pernah dibuat, sehingga tidak terbatas hanya satu objek seperti pada versi sebelumnya. `akunAktif` tetap dipertahankan, tetapi fungsinya berubah menjadi penunjuk ke rekening mana yang sedang aktif digunakan, bukan lagi satu-satunya tempat penyimpanan.

3. Ubah logika pada case 1 (buka rekening baru)

```
Rekening rekeningBaru = new Rekening(no, nama, saldo);
daftarRekening.add(rekeningBaru); // Tambahkan ke ArrayList, tidak menimpa yang lama
akunAktif= rekeningBaru; // Otomatis jadi akun aktif setelah dibuat
```

Pada versi sebelumnya, objek baru langsung ditampung ke akunAktif, sehingga rekening lama tertimpa setiap kali menu ini dipilih. Pada versi ini, objek yang baru dibuat terlebih dahulu ditampung ke variabel sementara rekeningBaru, kemudian dimasukkan ke daftarRekening menggunakan .add() agar data lama tetap tersimpan, dan baru setelah itu diset sebagai akunAktif agar rekening yang baru dibuat langsung dapat digunakan.

4. Tambahkan opsi menu baru “Ganti Akun”

```
System.out.println("5. Ganti Akun");
```

Ditambahkan pada tampilan daftar menu.

5. Tambahkan case 5 untuk logika Ganti Akun

```
case 5:
    if (daftarRekening.isEmpty()) {
        System.out.println("Error : Belum ada rekening yang terdaftar!");
    } else {
        System.out.print("Masukkan No Rekening yang ingin diaktifkan: ");
        String cariNo = input.nextLine();

        Rekening ditemukan = null;
        for (Rekening r : daftarRekening) {
            if (r.nomorRekening.equals(cariNo)) {
                ditemukan = r;
                break;
            }
        }

        if (ditemukan != null) {
            akunAktif = ditemukan;
            System.out.println("Berhasil beralih ke akun atas nama " + akunAktif.namaPemilik);
        } else {
            System.out.println("Error : Nomor rekening tidak ditemukan!");
        }
    }
    break;
```

Blok ini terdiri dari beberapa bagian logika:

- Pengecekan daftarRekening.isEmpty() di awal untuk memastikan minimal ada satu rekening yang bisa dipilih, sebelum meminta input nomor rekening.
- Perulangan for-each digunakan untuk menelusuri seluruh objek dalam daftarRekening satu per satu.

- Perbandingan `r.nomorRekening.equals(cariNo)` digunakan untuk mencocokkan nomor rekening yang dicari. Metode `.equals()` dipilih (bukan `==`) karena perbandingan dilakukan terhadap isi String, bukan referensi objeknya.
- Jika ditemukan kecocokan, objek tersebut disimpan ke variabel `ditemukan`, lalu di luar perulangan diset sebagai `akunAktif` yang baru.
- Jika tidak ditemukan kecocokan setelah seluruh data ditelusuri, program menampilkan pesan error.